



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks

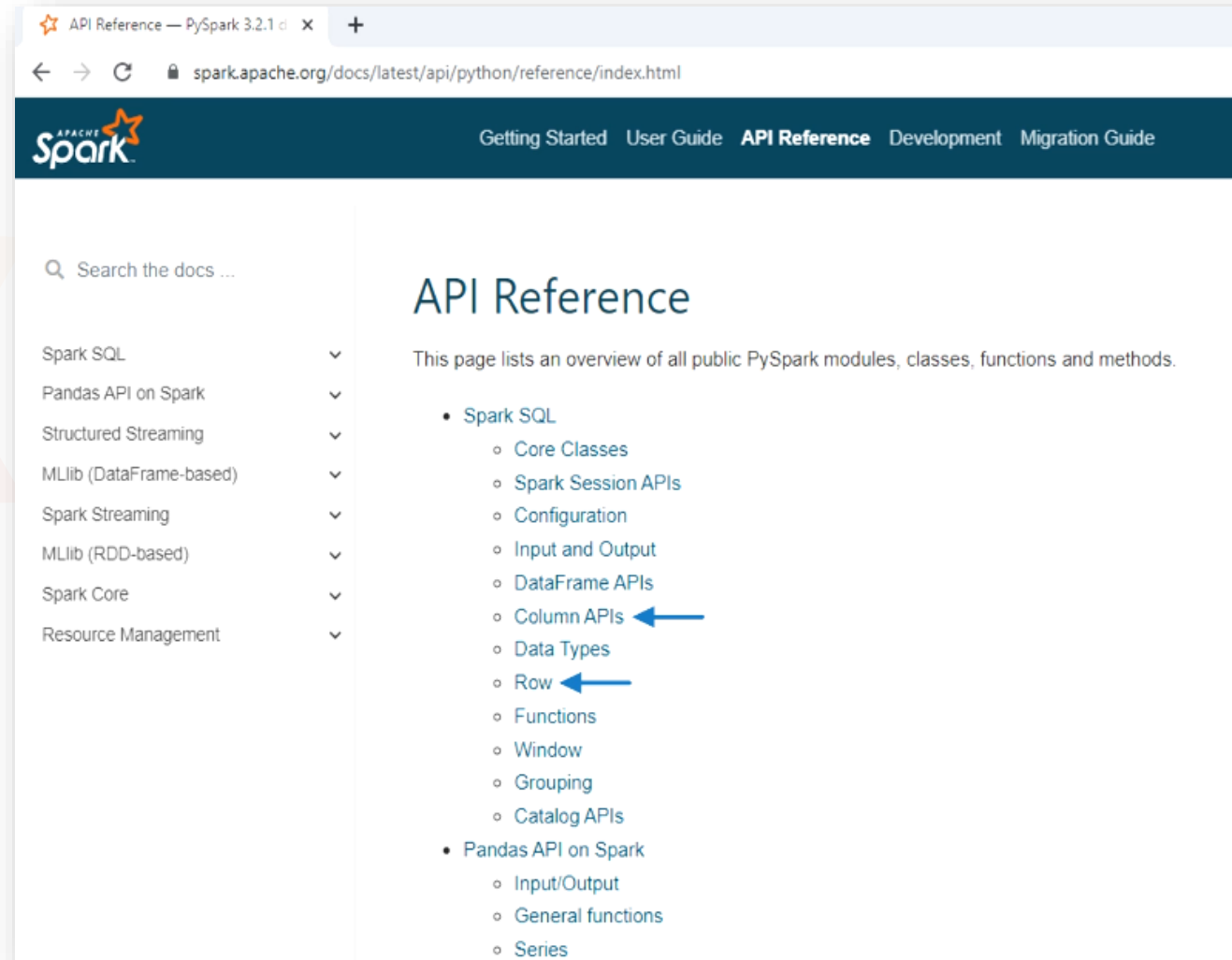


Spark Data Frame Columns

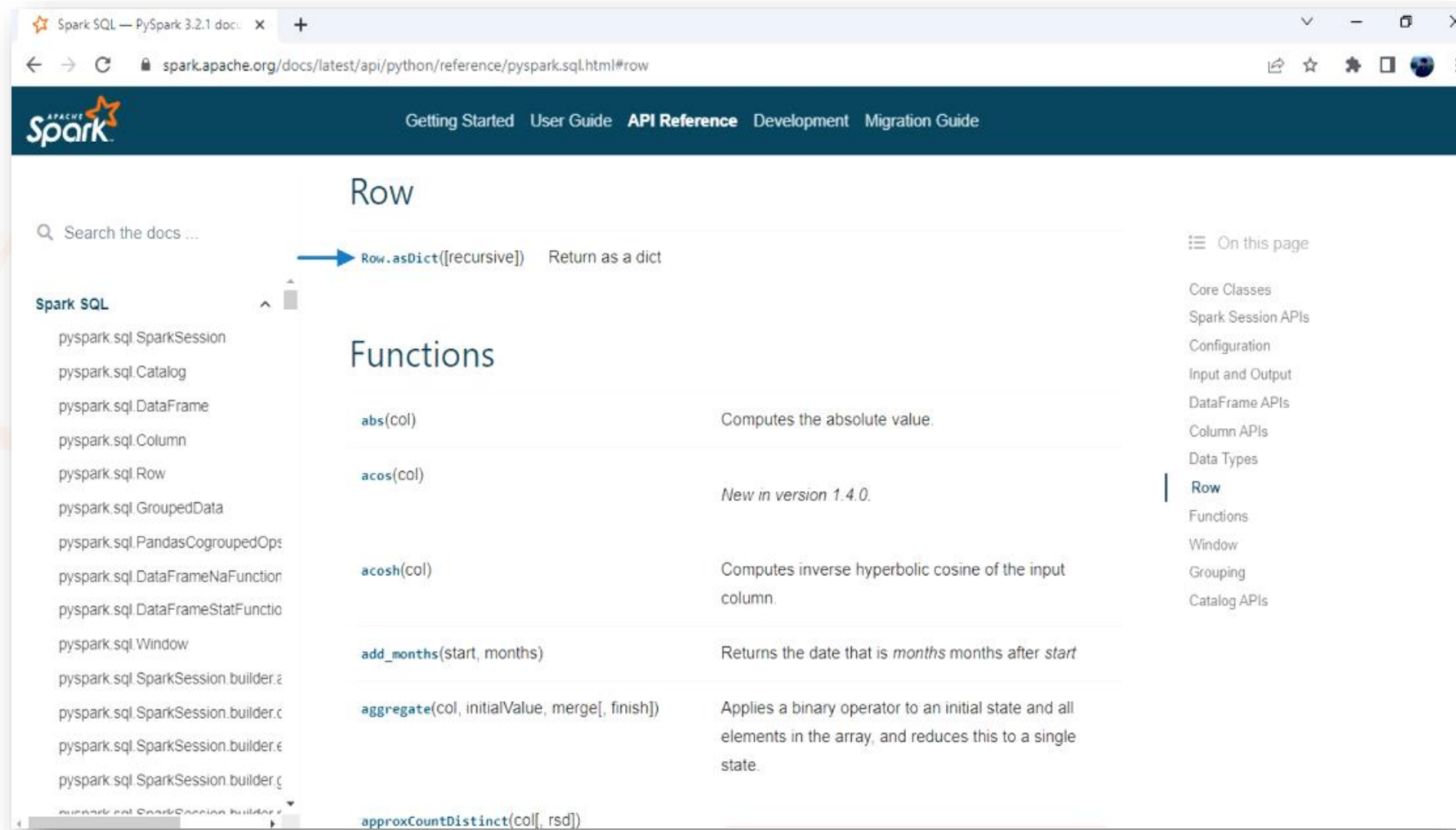
Spark also defined two objects to represent the Dataframe Row and column. Go to the latest Spark documentation - API reference page. **(Reference -**

<https://spark.apache.org/docs/latest/api/python/reference/index.html>)

You will see two links: Column APIs and Row



If you go to the link for row. The row is a simple class, and it gives you only one method: asDict(). The asDict() method converts a Row into a Python dictionary. You are not likely to directly work with the Row objects. We use the Row objects in writing unit test cases for the Spark application.



The screenshot shows the Apache Spark SQL API Reference page for the `Row` class. The page is titled "Row" and includes a search bar, a sidebar with a list of Spark SQL classes, and a main content area with a table of functions. A blue arrow points to the `Row.asDict([recursive])` method in the sidebar.

Row

Search the docs ...

Spark SQL

- pyspark.sql SparkSession
- pyspark.sql Catalog
- pyspark.sql DataFrame
- pyspark.sql Column
- pyspark.sql Row
- pyspark.sql GroupedData
- pyspark.sql PandasCgroupedOps
- pyspark.sql DataFrameNaFunction
- pyspark.sql DataFrameStatFunction
- pyspark.sql Window
- pyspark.sql SparkSession builder
- pyspark.sql SparkSession.builder
- pyspark.sql SparkSession.builder
- pyspark.sql SparkSession.builder

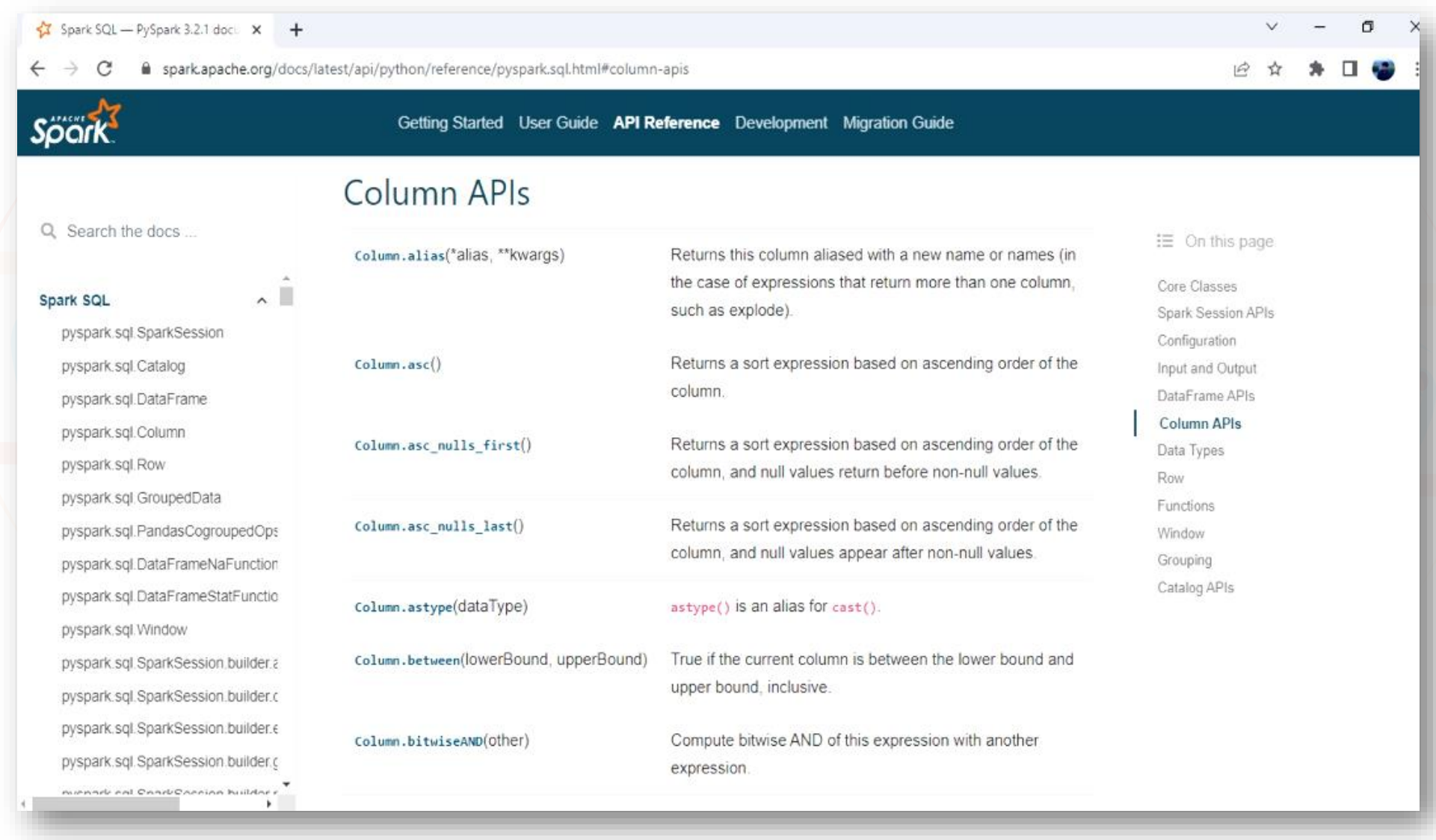
Functions

<code>abs(col)</code>	Computes the absolute value.
<code>acos(col)</code>	<i>New in version 1.4.0.</i>
<code>acosh(col)</code>	Computes inverse hyperbolic cosine of the input column.
<code>add_months(start, months)</code>	Returns the date that is <i>months</i> months after <i>start</i> .
<code>aggregate(col, initialValue, merge[, finish])</code>	Applies a binary operator to an initial state and all elements in the array, and reduces this to a single state.
<code>approxCountDistinct(col[, rsd])</code>	

On this page

- Core Classes
- Spark Session APIs
- Configuration
- Input and Output
- DataFrame APIs
- Column APIs
- Data Types
- Row**
- Functions
- Window
- Grouping
- Catalog APIs

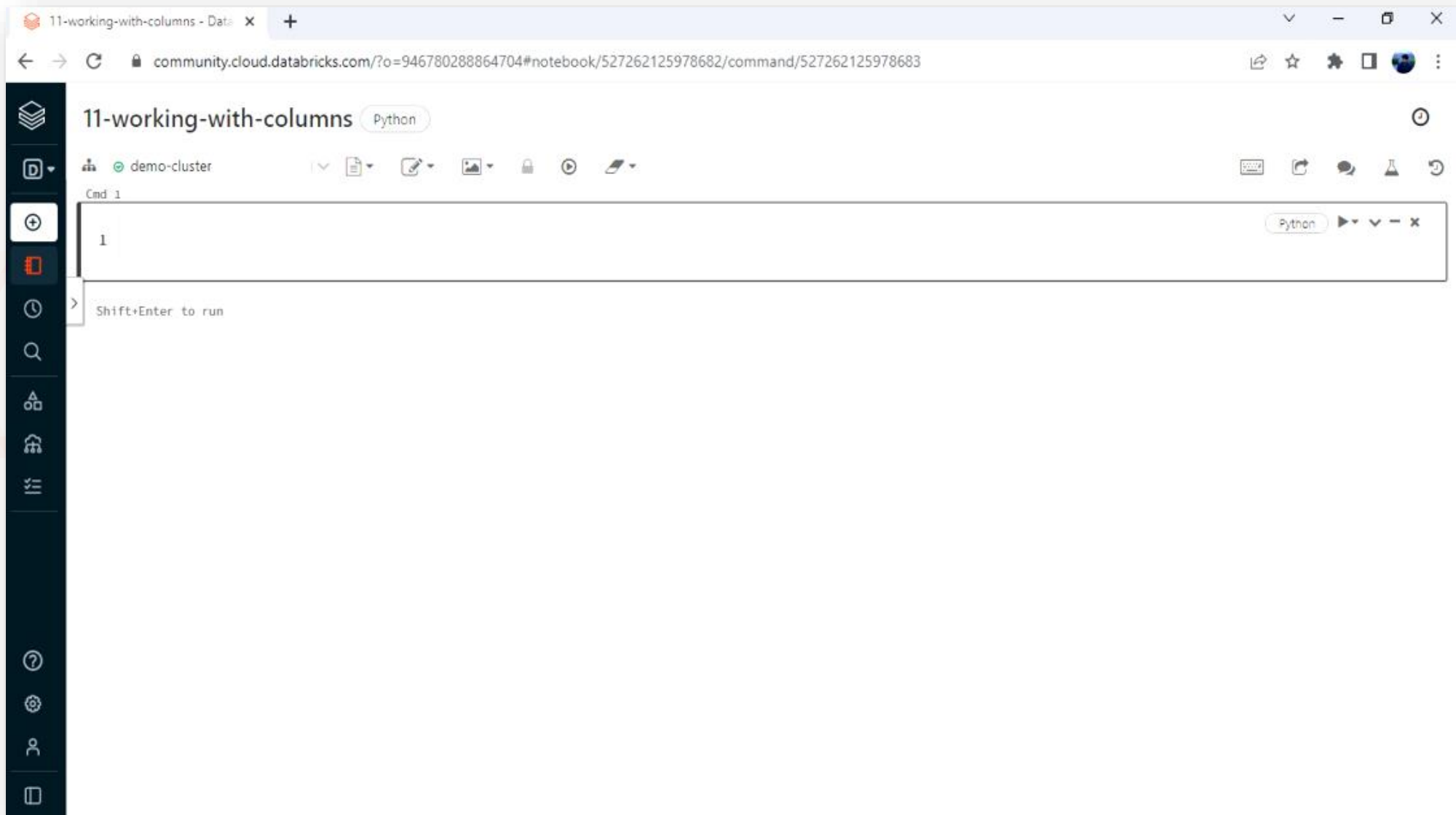
Spark Dataframe is made up of Row and Column objects. The row object is rudimentary, and we only use it in unit testing. Now let us go to the link for column API, and you will see the that the column object has got many methods.



Spark Dataframe is made up of Rows and Column objects. We do not have any row transformation in spark.
And most of the Dataframe transformations are all about transforming the columns.

Row	Column				
	Timestamp	Age	Gender	Country	state
	2014-08-27 11:29:31	37	Female	United States	IL
	2014-08-27 11:29:37	44	M	United States	IN
	2014-08-27 11:29:44	32	Male	Canada	
	2014-08-27 11:29:46	31	Male	United Kingdom	
	2014-08-27 11:30:22	31	Male	United States	TX

Log in to your Databricks community cloud and create a new notebook.(Reference : 11-working-with-columns)



I have created a Dataframe.

I am reading data using the spark.read. The format is CSV, and I am setting three options: Header, Infer Schema, and Sampling Ratio. Finally, I am loading the data into a Dataframe.

Infer schema option will read a sample of the data file and try to infer the schema automatically. The default value of the sample size is one percent. You can increase or decrease the sampling ratio. I am setting it to a smaller value to ensure that this command runs quickly.

```
11-working-with-columns Python
```

demo-cluster

```
1 airlines_df = spark.read \
2     .format("csv") \
3     .option("header", "true") \
4     .option("inferSchema", "true") \
5     .option("samplingRatio", "0.0001") \
6     .load("/databricks-datasets/airlines/part-00000")
7
8 display(airlines_df)
```

(3) Spark Jobs

	Year	Month	DayofMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	TailNum
1	1987	10	14	3	741	730	912	849	PS	1451	NA
2	1987	10	15	4	729	730	903	849	PS	1451	NA
3	1987	10	17	6	741	730	918	849	PS	1451	NA
4	1987	10	18	7	729	730	847	849	PS	1451	NA
5	1987	10	19	1	749	730	922	849	PS	1451	NA
6	1987	10	21	3	728	730	848	849	PS	1451	NA
7	1987	10	22	4	728	730	852	849	PS	1451	NA

Truncated results, showing first 1000 rows.

Command took 14.52 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:12:46 PM on demo-cluster

So we have an origin, destination, and distance columns.
We also have Arrival delays and departure delays before the origin column.
I want to select only these five columns.

11-working-with-columnsPython

demo-cluster

Cmd 1

```
1 airlines_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .option("samplingRatio", "0.0001") \  
6     .load("/databricks-datasets/airlines/part-00000") \  
7 \  
8 display(airlines_df)
```

(3) Spark Jobs

	usedTime	AirTime	ArrDelay	DepDelay	Origin	Dest	Distance	TaxiIn	TaxiOut	Cancelled	CancellationCode	Divert
1		NA	23	11	SAN	SFO	447	NA	NA	0	NA	0
2		NA	14	-1	SAN	SFO	447	NA	NA	0	NA	0
3		NA	29	11	SAN	SFO	447	NA	NA	0	NA	0
4		NA	-2	-1	SAN	SFO	447	NA	NA	0	NA	0
5		NA	33	19	SAN	SFO	447	NA	NA	0	NA	0
6		NA	-1	-2	SAN	SFO	447	NA	NA	0	NA	0
7												

Truncated results, showing first 1000 rows.

Command took 14.52 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:12:46 PM on demo-cluster

Here is the code to select the desired 5 columns. And we can see the output for the same given below.

11-working-with-columns Python

demo-cluster

Cmd 2

```
1 airlines_df.select("Origin", "Dest", "Distance", "ArrDelay", "DepDelay") \
2     .show(5)
```

>

(1) Spark Jobs

Origin	Dest	Distance	ArrDelay	DepDelay
SAN	SFO	447	23	11
SAN	SFO	447	14	-1
SAN	SFO	447	29	11
SAN	SFO	447	-2	-1
SAN	SFO	447	33	19

only showing top 5 rows

Command took 0.98 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:15:59 PM on demo-cluster

We have five approaches to referring to a column name.

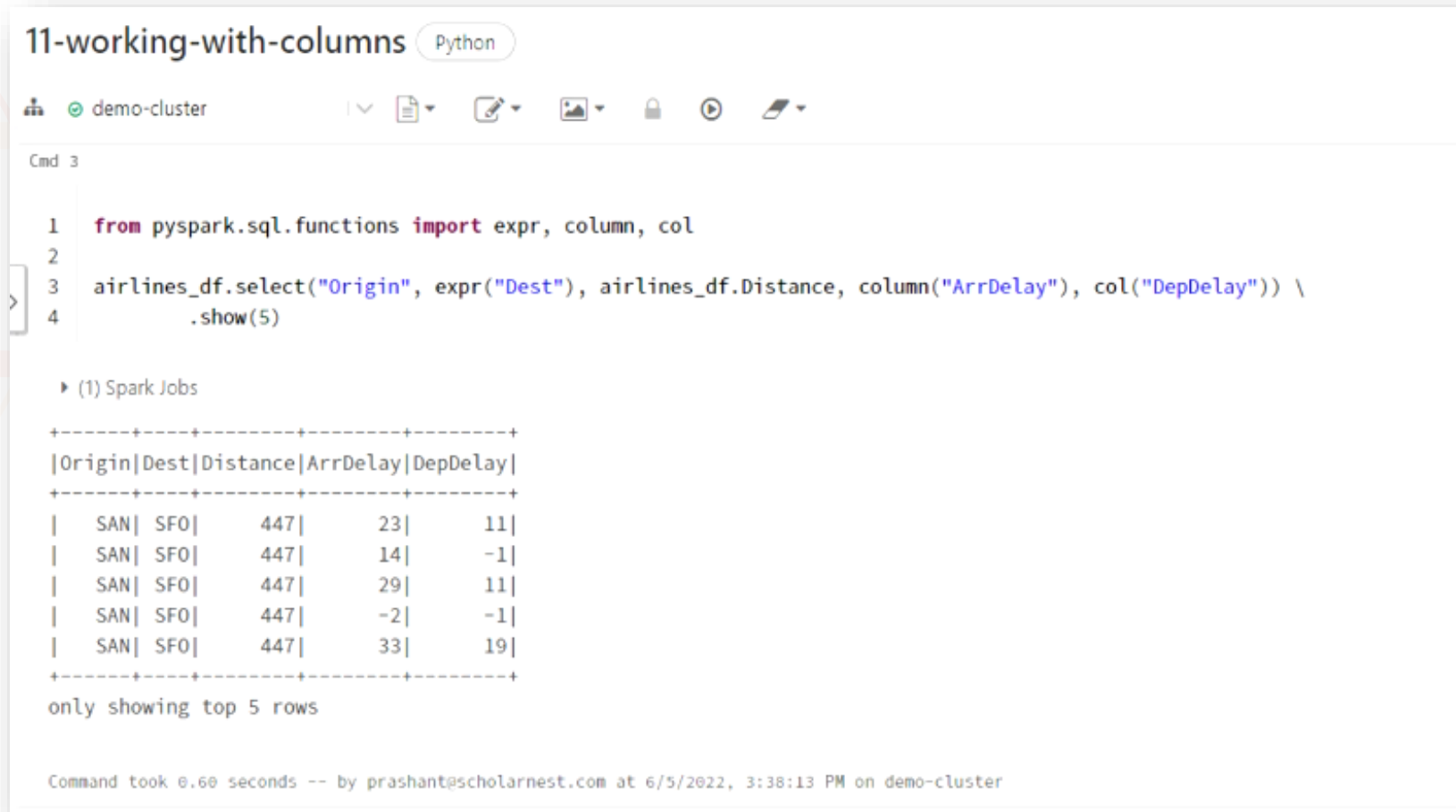
1. "Origin"
2. `expr("Origin")`
3. `airlines_df.Origin`
4. `column("Origin")`
5. `col("Origin")`

The first approach is a column name string. The second approach uses the `expr()` function and the column name. The third one is using a dataframe.column name. The fourth approach is using a `column()` function. The last one is using the `col()` function.

The fourth and the fifth approaches are the same because the `col()` function is an alias for the `column()` function. So the fourth and the fifth approaches are the same.

But officially, we have five different approaches to refer to a column name. You can use any of these approaches in your select expression.

Here is the code to select the desired 5 columns. But I am referring to each column differently. I am referring to the Origin column using a plain string name. But I am referring to the destination using the `expr()` function. Similarly, I am referring to the distance using the `dataframe.column` name. The next one is using the `column()` function. And the last one is using the `col()` function.



The screenshot shows a Jupyter Notebook interface with the title "11-working-with-columns" and a "Python" language selector. The notebook is running on a "demo-cluster". The code cell, labeled "Cmd 3", contains the following Python code:

```
1 from pyspark.sql.functions import expr, column, col
2
3 airlines_df.select("Origin", expr("Dest"), airlines_df.Distance, column("ArrDelay"), col("DepDelay")) \
4     .show(5)
```

Below the code, the output shows "(1) Spark Jobs" and a table of the first 5 rows of the selected data:

Origin	Dest	Distance	ArrDelay	DepDelay
SAN	SFO	447	23	11
SAN	SFO	447	14	-1
SAN	SFO	447	29	11
SAN	SFO	447	-2	-1
SAN	SFO	447	33	19

Below the table, it says "only showing top 5 rows". At the bottom, a status bar indicates: "Command took 0.60 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:38:13 PM on demo-cluster".

1. "Origin"
2. `expr("Origin")`
3. `airlines_df.Origin`
4. `column("Origin")`
5. `col("Origin")`

The first approach is for simplicity. I prefer using the first approach and simply writing the column name string. It is easy to use and looks neat.

But the first approach doesn't allow writing expressions.
And that's why we have the second approach of using the `expr()` function.

I am selecting three columns here in the code shown below. The first two columns are Origin and Destination. These are simple names, so they work. But the third column is a SQL expression. I am multiplying distance by 1.6 to convert it from miles to kilometers. So the third column is not a simple column name, it is an expression. You will see an error if you run it. And that's why we have the `expr()` function.



The screenshot shows a Jupyter Notebook window titled "11-working-with-columns" with a "Python" language selector. The interface includes a toolbar with icons for file operations, a search bar, and a command palette. A red error message is displayed at the top right: "Error running command 4. Go to command". Below this, a status bar indicates "Command took 0.60 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:38:13 PM on demo-cluster". The command area shows "Cmd 4" with the following Python code:

```
1 airlines_df.select("Origin", "Dest", "Distance * 1.6 as km_distance") \
2     .show(5)
```

The output area displays an `AnalysisException` error: "Column 'Distance * 1.6 as km_distance' does not exist. Did you mean one of the following? [Distance, CancellationCode, Cancelled, Diverted, UniqueCarrier, ActualElapsedTime, CRSElapsedTime, DepTime, Dest, IsArrDelayed, IsDepDelayed, LateAircraftDelay, AirTime, CRSDepTime, DayOfWeek, DayOfMonth, DepDelay, Origin, TaxiIn, WeatherDelay, ArrDelay, ArrTime, CRSArrTime, CarrierDelay, FlightNum, Month, NASDelay, SecurityDelay, TailNum, TaxiOut, Year];". Below the error, the project and relation schemas are shown:

```
'Project [Origin#89, Dest#90, 'Distance * 1.6 as km_distance]
+- Relation [Year#73,Month#74,DayOfMonth#75,DayOfWeek#76,DepTime#77,CRSDepTime#78,ArrTime#79,CRSArrTime#80,UniqueCarrier#81,FlightNum#82,TailNum#83,ActualElapsedTime#84,CRSElapsedTime#85,AirTime#86,ArrDelay#87,DepDelay#88,Origin#89,Dest#90,Distance#91,TaxiIn#92,TaxiOut#93,Cancelled#94,CancellationCode#95,Diverted#96,... 7 more fields] csv
```

The status bar at the bottom indicates "Command took 0.50 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:39:49 PM on demo-cluster". The command area shows "Cmd 5" with no visible code.

It works with `expr()` function.

11-working-with-columns Python

demo-cluster

Cmd 4

```
1 airlines_df.select("Origin", "Dest", expr("Distance * 1.6 as km_distance")) \
2     .show(5)
```

▶ (1) Spark Jobs

Origin	Dest	km_distance
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2

only showing top 5 rows

Command took 0.83 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:40:20 PM on demo-cluster

So we learned the need for the first two approaches.

1. "Origin" -> Used for plain column names and simplicity
2. `expr("Distance * 1.6 as km_distance")` -> Used to evaluate SQL like expression strings

The first one is a simple column name, and we use it for simplicity.

But when you have an expression, you must evaluate it using the `expr()` function.

So the second approach is for evaluating SQL-like expression strings.

The `expr()` function also works fine for simple column names. But we do not use it for column names.

We use the `expr()` function when we have an expression.

We have the following six fields in this given Dataframe as shown in the right hand side of the screenshot. I want to write a select expression to produce the following output shown in the left side of the screenshot.

1. UniqueCarrier

2. FlightNum

3. Year

4. Month

5. DayofMonth

6. Distance

UniqueCarrier	FlightNum	FlightDate	DistanceKM
PS	1451	1987-10-14	715.2
PS	1451	1987-10-15	715.2
PS	1451	1987-10-17	715.2
PS	1451	1987-10-18	715.2
PS	1451	1987-10-19	715.2

Here is one possible solution. The first and the second columns are simply the names. So I am not using the `expr()` function for those column names. But the third and the fourth columns are expressions. So I am using the `expr()` function.



```
1 airlines_df.select("Origin",
2                     "FlightNum",
3                     expr("Year||'-'||Month||'-'||DayOfMonth as FlightDate"),
4                     expr("Distance * 1.6 as DistanceKM")) \
5                     .show(5)
```

▶ (1) Spark Jobs

Origin	FlightNum	FlightDate	DistanceKM
SAN	1451	1987-10-14	715.2
SAN	1451	1987-10-15	715.2
SAN	1451	1987-10-17	715.2
SAN	1451	1987-10-18	715.2
SAN	1451	1987-10-19	715.2

only showing top 5 rows

Command took 0.83 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:43:55 PM on demo-cluster

The third approach is using the dataframe.column name. We use the third approach to avoid column name ambiguity. We have another Dataframe airport_df shown below. The airlines_df is the main dataframe, and the airport_df is a lookup data frame. And I want to produce results like the one shown in the screenshot below.

11-working-with-columnsPython

demo-cluster

Command took 0.83 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:43:55 PM on demo-cluster

Cmd 6

```
1 data_list = [("SAN", "San Antonio"),
2             ("SFO", "San Francisco"),
3             ("BUR", "Hollywood Burbank"),
4             ("OAK", "Oakland")]
5
6 airport_df = spark.createDataFrame(data_list).toDF("Origin", "OriginAirportName")
7 display(airport_df)
```

(3) Spark Jobs

	Origin	OriginAirportName
1	SAN	San Antonio
2	SFO	San Francisco
3	BUR	Hollywood Burbank
4	OAK	Oakland

Showing all 4 rows.

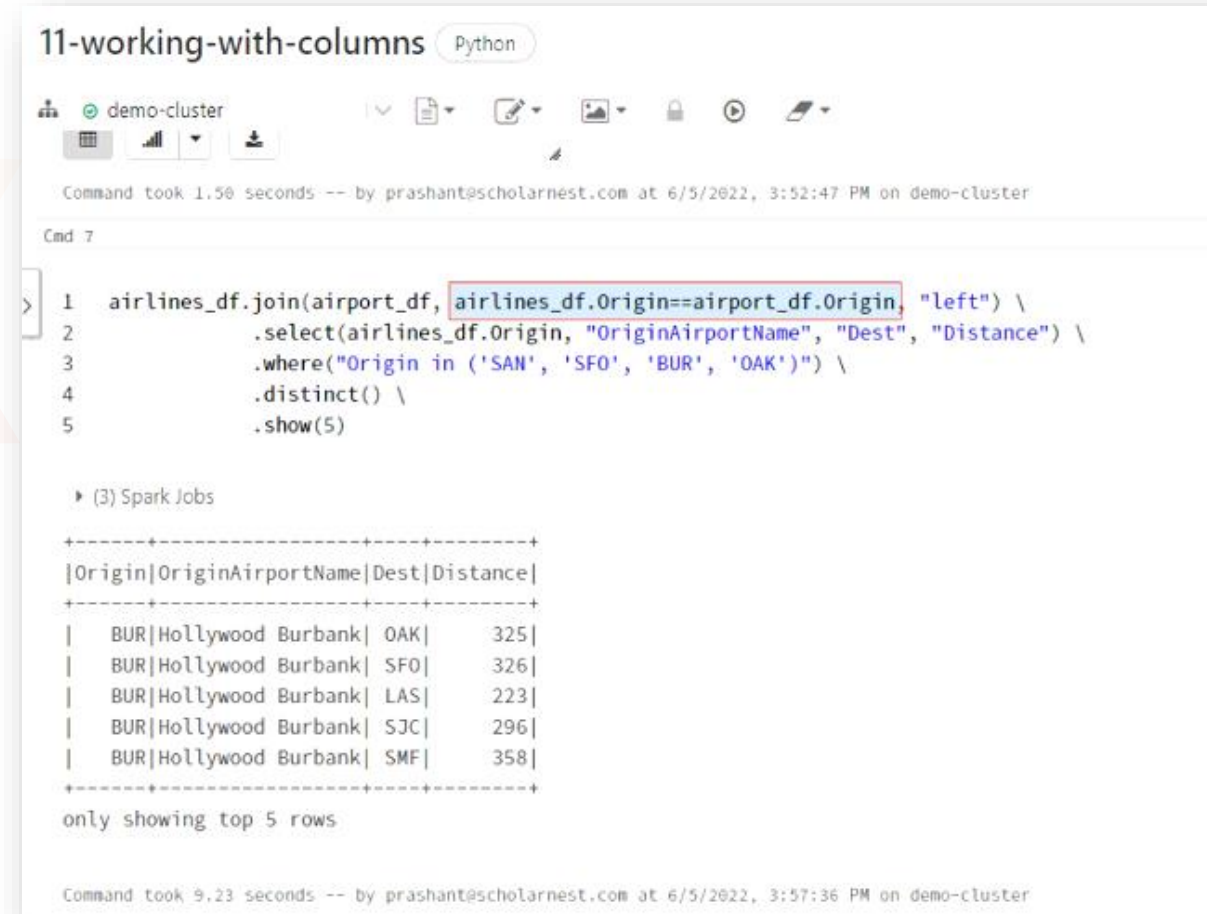
Origin	OriginAirportName	Dest	Distance
BUR	Hollywood Burbank	OAK	325
BUR	Hollywood Burbank	SFO	326
BUR	Hollywood Burbank	LAS	223
BUR	Hollywood Burbank	SJC	296
BUR	Hollywood Burbank	SMF	358

Command took 1.50 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:52:47 PM on demo-cluster

Cmd 7

We can use the following code to achieve the desired result. We are doing a left join of airlines_df with the airport_df. Look at the join condition highlighted below.

We have a small problem with the airlines_df and the airport_df. They both have the same column name. The column name is the origin of both the data frames. And we cannot say Origin==Origin, Spark will complain about ambiguous column names. So we could use the dataframe.column name to avoid column name ambiguity.



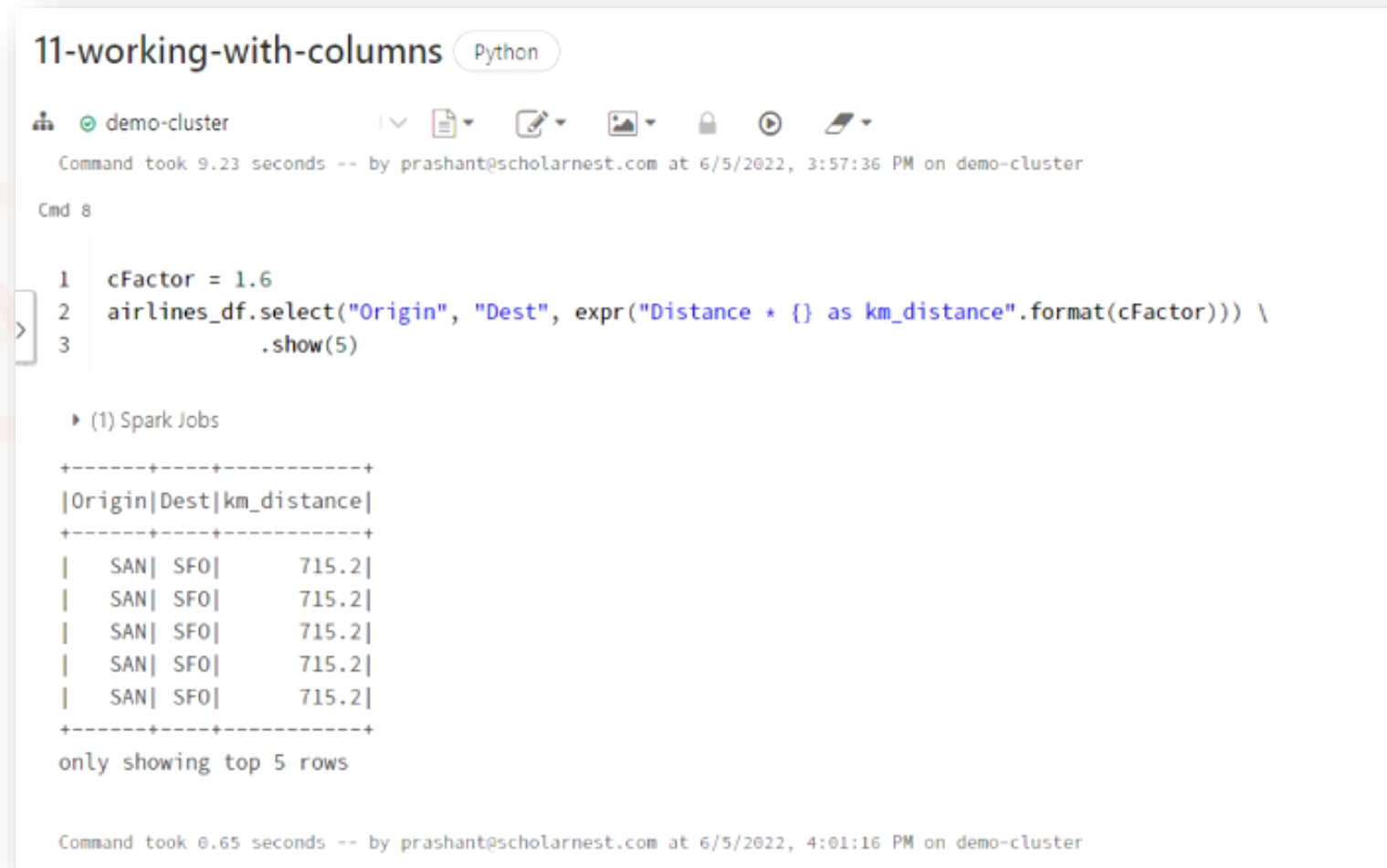
```
11-working-with-columns Python
demo-cluster
Command took 1.50 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:52:47 PM on demo-cluster

Cmd 7
> 1 airlines_df.join(airport_df, airlines_df.Origin==airport_df.Origin, "left") \
2   .select(airlines_df.Origin, "OriginAirportName", "Dest", "Distance") \
3   .where("Origin in ('SAN', 'SFO', 'BUR', 'OAK')") \
4   .distinct() \
5   .show(5)

(3) Spark Jobs
+-----+-----+-----+-----+
|Origin|OriginAirportName|Dest|Distance|
+-----+-----+-----+-----+
| BUR|Hollywood Burbank| OAK|      325|
| BUR|Hollywood Burbank| SFO|      326|
| BUR|Hollywood Burbank| LAS|      223|
| BUR|Hollywood Burbank| SJC|      296|
| BUR|Hollywood Burbank| SMF|      358|
+-----+-----+-----+-----+
only showing top 5 rows

Command took 9.23 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:57:36 PM on demo-cluster
```

Here is an expression that we created earlier. I have a cFactor variable. And I am using the variable inside the expression using Python string substitution technique. But instead of using this string technique, some people prefer to use the col() function.



The screenshot shows a Databricks notebook interface. At the top, the title is "11-working-with-columns" with a "Python" language selector. Below the title, it says "demo-cluster" and "Command took 9.23 seconds -- by prashant@scholarnest.com at 6/5/2022, 3:57:36 PM on demo-cluster". The command area shows "Cmd 8" with the following Python code:

```
1 cFactor = 1.6
2 airlines_df.select("Origin", "Dest", expr("Distance * {} as km_distance".format(cFactor))) \
3     .show(5)
```

Below the code, it says "▶ (1) Spark Jobs". The output is a table with 5 rows and 3 columns: Origin, Dest, and km_distance. The data is as follows:

Origin	Dest	km_distance
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2

Below the table, it says "only showing top 5 rows". At the bottom, it says "Command took 0.65 seconds -- by prashant@scholarnest.com at 6/5/2022, 4:01:16 PM on demo-cluster".

Here is the same code using the col() function.

11-working-with-columns Python

demo-cluster

Cmd 9

```
1 cFactor = 1.6
2 airlines_df.select("Origin", "Dest", col("Distance") * cFactor) \
3     .show(5)
```

▶ (1) Spark Jobs

Origin	Dest	(Distance * 1.6)
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2
SAN	SFO	715.2

only showing top 5 rows

Command took 0.48 seconds -- by prashant@scholarnest.com at 6/5/2022, 4:02:26 PM on demo-cluster

Here is a Dataframe select statement. I am selecting five columns and using five different approaches to select the columns. The first one is a column name string. So place your cursor after the string name, type a dot symbol and press the tab. You will see a bunch of string functions such as capitalize, case fold, center, etc. These are all Python string manipulation functions.

11-working-with-columns Python

demo-cluster

SAN| SFO| 715.2|

+-----+

only showing top 5 rows

Command took 0.48 seconds -- by prashant@scholarnest.com at 6/5/2022, 4:02:26 PM on demo-cluster

> Cmd 10

1 airlines_df.select("Origin").

2 expr("Des

3 airlines_

4 column("A

5 col("DepD

6 .show(5)

capitalize

casefold

center

count

encode

endswith

expandtabs

find

format_map

format

index

isalnum

isalpha

Shift+Enter to run

But if you try getting a similar lookup for other approaches, you will see column APIs. Place your cursor at the end of the `expr()` function, type a dot, and press the tab. You will see column APIs such as `alias`, `asc`, `between`, etc. These are column APIs, and they help you create complex expressions.

11-working-with-columns Python

demo-cluster
SAN SFO 715.2
only showing top 5 rows

Command took 0.48 seconds -- by prashant@scholarnest.com at 6/5/2022, 4:02:26 PM on demo-cl

> Cmd 10

```
1 airlines_df.select("Origin",
2                     expr("Dest").,
3                     airlines_df.D
4                     column("ArrDe
5                     col("DepDelay
6                     .show(5)
```

alias
asc_nulls_first
asc_nulls_last
asc
astype
between
bitwiseAND
bitwiseOR
bitwiseXOR
cast
contains
desc_nulls_first
desc nulls last

Shift+Enter to run

So I am selecting five columns here in this example. Then I am applying a where() condition. I wanted to apply two conditions.

- 1. where the origin is like B%
- 2. ArrDelay < 0

I defined both the conditions here. But I implement it using the Dataframe Column APIs. The col("Origin") will give me a Dataframe column object. The like() is a Column API. Similarly, I defined the second condition. This approach to defining expressions might look complicated for many developers. So we often do it using string expressions.

11-working-with-columns Python

demo-cluster

Command took 0.48 seconds -- by prashant@scholarnest.com at 6/5/2022, 4:02:26 PM on demo-cluster

Cmd 10

```
1 airlines_df.select("Origin", "Dest", "Distance", "ArrDelay", "DepDelay") \
2   .where((col("Origin").like("B%")) &
3         (col("ArrDelay") < 0)) \
4   .show(5)
```

(1) Spark Jobs

Origin	Dest	Distance	ArrDelay	DepDelay
BUR	OAK	325	-6	-1
BUR	OAK	325	-1	0
BUR	OAK	325	-6	0
BUR	OAK	325	-2	-1
BUR	OAK	325	-3	0

only showing top 5 rows

Command took 1.08 seconds -- by prashant@scholarnest.com at 6/7/2022, 2:06:02 AM on demo-cluster

Here is the same example but using the string expression. Look at the string expression here. This approach looks easy for SQL developers. The earlier approach looks easy for Python or Java Experts. But you have both options. Feel free to use any of the approaches, and Spark performs the same for both approaches. I am using the `expr()` function inside the `where()` method. But the `expr()` is options inside the `where()` method.

11-working-with-columns Python

demo-cluster only showing top 5 rows

Command took 1.08 seconds -- by prashant@scholarnest.com at 6/7/2022, 2:06:02 AM on demo-cluster

Cmd 11

```
1 airlines_df.select("Origin", "Dest", "Distance", "ArrDelay", "DepDelay") \
2     .where(expr("Origin like 'B%' and ArrDelay < 0"))
3     .show(5)
```

(1) Spark Jobs

Origin	Dest	Distance	ArrDelay	DepDelay
BUR	OAK	325	-6	-1
BUR	OAK	325	-1	0
BUR	OAK	325	-6	0
BUR	OAK	325	-2	-1
BUR	OAK	325	-3	0

only showing top 5 rows

Command took 0.78 seconds -- by prashant@scholarnest.com at 6/7/2022, 2:07:45 AM on demo-cluster

The `where()` method expects an expression, and it will automatically evaluate the string. So you can remove the `expr()` function and simply use the string.

11-working-with-columns Python

demo-cluster
only showing top 5 rows

Command took 1.08 seconds -- by prashant@scholarnest.com at 6/7/2022, 2:06:02 AM on demo-cluster

Cmd 11

```
1 airlines_df.select("Origin", "Dest", "Distance", "ArrDelay", "DepDelay") \
2     .where("Origin like 'B%' and ArrDelay < 0") \
3     .show(5)
```

► (1) Spark Jobs

Origin	Dest	Distance	ArrDelay	DepDelay
BUR	OAK	325	-6	-1
BUR	OAK	325	-1	0
BUR	OAK	325	-6	0
BUR	OAK	325	-2	-1
BUR	OAK	325	-3	0

only showing top 5 rows

Command took 0.86 seconds -- by prashant@scholarnest.com at 6/7/2022, 2:08:25 AM on demo-cluster



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com